

Course Outline: Docker Compose

Title: Docker Compose: Orchestrating Multi-Container Applications **Skill:** Skill 34: Construir aplicaciones en contenedores **Learnpack:** + Docker Composer **File:** s34-w12d35-docker-composer **Prerequisite:** Docker (s34-w12d35-docker) **Target Audience:** Beginner developers who know Docker basics (images, containers, Dockerfile) and are ready to manage multi-container applications **Total Lessons:** 12 **Format:** Mixed (8% hands-on = 1 lesson)

Course Description

Running a single container is straightforward — but real applications are rarely just one service. A web app needs a database, maybe a cache, maybe a message queue. Managing all of those containers by hand gets unwieldy fast. Docker Compose solves this: it lets you define an entire multi-service environment in a single YAML file and bring it all up with one command. In this course, students learn how Compose works, how to configure a real multi-service project, and how a well-structured Dockerized application is organized from top to bottom.

Course Philosophy

This course emphasizes:

- Starting from the problem, not the tool — students understand *why* Compose exists before learning how to use it
 - Building confidence through a real, runnable project — the hands-on challenge produces observable output
 - Best practices woven naturally into context — not isolated rules, but habits that emerge from understanding the reasoning
-

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - Docker Compose Fundamentals	3
02 - Writing docker-compose.yml	3
03 - Project Architecture	2
04 - Assessment	2
05 - Outro	1
Total	12

00.0 Welcome to Docker Compose

Learning Objectives:

- Understand what problem Docker Compose solves and why it matters
- Know what the course covers and how each section builds on the last

Content Outline:

- The multi-container reality: most real apps need more than one service
- What Docker Compose is and what it does for you
- What students will be able to do by the end of this course
- Brief tour of the course sections

Transitions: This lesson motivates the course. Section 01 begins by going deeper into *how* Compose solves the multi-container problem.

Key Principles:

- Real applications are systems, not single containers
- Compose trades manual setup for a declarative configuration file

Examples:

- A web app that needs a web server container, a PostgreSQL container, and a Redis container — starting all three manually vs. one `docker compose up`
-

01.0 Docker Compose Fundamentals

Learning Objectives:

- Articulate what Docker Compose is and what problem it solves
- Identify the three core concepts in any Compose setup: services, ports, and networking
- Understand the role of environment variables in a Compose project

Content Outline:

- The multi-container problem in concrete terms
- What Docker Compose is: a tool for defining and running multi-container applications
- The `docker-compose.yml` file as the single source of truth for your environment
- Overview of the three sub-lessons in this section: services/ports/networking, then environment variables
- Key commands: `docker compose up`, `docker compose down`, `docker compose logs`

Transitions: This lesson frames the section. Lesson 01.1 dives into services, ports, and networking — the structural building blocks of any Compose file.

Key Principles:

- Compose replaces a series of `docker run` commands with a single declarative file

- The `docker-compose.yml` file describes your system, not the steps to build it

Examples:

- Before Compose: `docker run -d -p 5432:5432 --name db postgres` + `docker run -d -p 3000:3000 --link db myapp` — fragile, hard to remember, easy to get wrong
 - After Compose: `docker compose up` — one command, all services, correct networking automatically
-

01.1 Services, Ports, and Networking

Learning Objectives:

- Define a service in a `docker-compose.yml` file
- Configure port mappings between container and host
- Explain how containers in the same Compose project communicate with each other
- Apply the lightweight image best practice when choosing base images

Content Outline:

- Services: what they are and how each maps to a running container
- The `image` and `build` keys — using a pre-built image vs. building from a Dockerfile
- Port mapping: `HOST_PORT:CONTAINER_PORT` — why both sides matter
- Container networking in Compose: services share a default network and resolve each other by service name
- Lightweight images: why `node:20-alpine` is preferred over `node:20` for production-like setups

Transitions: Services, ports, and networking are the skeleton of a Compose file. Lesson 01.2 adds the configuration layer: environment variables.

Key Principles:

- In Compose, services communicate using their service name as the hostname — not `localhost`
- Port mapping exposes container ports to the host; without it, the service is only reachable by other containers
- Lightweight base images reduce attack surface and pull/build time

Examples:

- A `web` service using `build: .` (local Dockerfile) and a `db` service using `image: postgres:15-alpine`
- Port mapping: `"3000:3000"` on the `web` service — accessible at `http://localhost:3000`
- The `web` service connects to the database at `db:5432`, not `localhost:5432`

Anti-pattern — Port Conflict:

- Mapping the same host port to two different services (e.g., two services both mapped to `"3000:3000"`) causes a startup failure
- Why it's tempting: copy-pasting service definitions without adjusting ports

- What goes wrong: `Bind for 0.0.0.0:3000 failed: port is already allocated`
 - Fix: each service must map to a unique host port
-

01.2 Environment Variables

Learning Objectives:

- Pass environment variables to services using the `environment` key
- Use a `.env` file with Compose to separate configuration from code
- Explain the `env_file` directive and when to use it
- Apply the `.env` best practice for sensitive configuration

Content Outline:

- Why environment variables matter in containerized apps: configuration changes between environments (dev, staging, prod)
- The `environment` key: inline variable definitions
- The `.env` file: Compose automatically reads it from the project root and substitutes `${VAR}` references
- The `env_file` directive: loading a named file of variables directly into a service
- Best practice: secrets and configuration go in `.env`, never hardcoded in `docker-compose.yml`
- `.env` in `.gitignore`: never commit secrets

Transitions: Environment variables give services the configuration they need at runtime. Section 02 applies everything from this section by writing a real `docker-compose.yml` from scratch.

Key Principles:

- The `.env` file is for the Compose file itself (variable substitution); `env_file` is for passing variables into a container
- Never hardcode secrets — passwords, API keys, and tokens belong in `.env`

Examples:

- Inline: `environment: - POSTGRES_PASSWORD=secret` — works, but leaks secrets into version control
- Better: `environment: - POSTGRES_PASSWORD=${DB_PASSWORD}` + a `.env` file with `DB_PASSWORD=secret`
- `env_file: - .env.db` — loads all variables from `.env.db` into the container's environment

Anti-pattern — Missing Variable:

- Referencing `${DB_PASSWORD}` in `docker-compose.yml` without defining it in `.env`
 - What goes wrong: the variable resolves to an empty string — no error at startup, but the service behaves incorrectly
 - Fix: always define every `${VAR}` reference in `.env`; use `docker compose config` to inspect resolved values
-

02.0 Writing docker-compose.yml

Learning Objectives:

- Identify the top-level keys of a `docker-compose.yml` file
- Understand how the file structure maps to running containers
- Know the commands to start, stop, and inspect a Compose project

Content Outline:

- Anatomy of `docker-compose.yml`: `services`, `networks`, `volumes` (top-level keys)
- The `services` block: where every container is defined
- Compose commands: `up`, `down`, `ps`, `logs`, `exec`
- The `--build` flag: when to force a rebuild
- Preview of what Lessons 02.1 and 02.2 will build

Transitions: This lesson maps the YAML structure to the mental model from Section 01. Lesson 02.1 writes a real compose file for a web app + database project.

Key Principles:

- `docker compose up -d` starts everything in detached mode; `docker compose down` stops and removes containers
- `docker compose logs -f [service]` is your first debugging tool when something doesn't start correctly

Examples:

- Minimal valid `docker-compose.yml` with one service: `services: web: image: nginx`
 - The `--build` flag: `docker compose up --build` — necessary after changing a Dockerfile
-

02.1 Configuring a Multi-Service Project

Learning Objectives:

- Write a complete `docker-compose.yml` for a web application backed by a database
- Configure service dependencies so the database starts before the web app
- Apply `.dockerignore` to avoid sending unnecessary files to the build context
- Identify and fix the most common configuration errors

Content Outline:

- Step-by-step: defining a `web` service (built from local Dockerfile) and a `db` service (PostgreSQL image)
- The `depends_on` key: controlling startup order
- Connecting the web service to the database using the service name as hostname
- `.dockerignore`: what to exclude from the build context (`node_modules`, `.git`, `.env`) and why
- Common configuration errors and how to fix them

Transitions: This lesson walks through the full configuration. Lesson 02.2 turns the student loose to write it themselves.

Key Principles:

- `depends_on` controls startup order but does not wait for readiness — the app must handle connection retries
- `.dockerignore` is essential: sending `node_modules` to the build context can make builds 10× slower

Examples:

```
services:
  web:
    build: .
    ports:
      - "3000:3000"
    environment:
      - DATABASE_URL=postgres://postgres:${DB_PASSWORD}@db:5432/mydb
    depends_on:
      - db
  db:
    image: postgres:15-alpine
    environment:
      - POSTGRES_PASSWORD=${DB_PASSWORD}
      - POSTGRES_DB=mydb
```

Anti-pattern — Hardcoded Service URL:

- Writing `DATABASE_URL=postgres://postgres:secret@localhost:5432/mydb` inside the container
- Why it's tempting: it's the same URL used when running locally without Docker
- What goes wrong: `localhost` inside the container refers to the container itself, not the database service
- Fix: use the service name (`db`) as the hostname — `@db:5432`

Anti-pattern — No `.dockerignore`:

- Sending the full project directory (including `node_modules`) as the build context
- What goes wrong: builds are slow and bloated; local `node_modules` can overwrite the image's installed packages
- Fix: add `node_modules`, `.git`, `.env`, and build artifacts to `.dockerignore`

02.2 Multi-Service Application

Challenge Prompt: Write a `docker-compose.yml` that starts a Node.js web service and a PostgreSQL database, where the web service connects to the database using the service name as hostname and the database password is read from a `.env` file.

Solution Code:

```
# docker-compose.yml
services:
  web:
    build: .
    ports:
      - "3000:3000"
    environment:
      - DATABASE_URL=postgres://postgres:${DB_PASSWORD}@db:5432/appdb
    depends_on:
      - db

  db:
    image: postgres:15-alpine
    environment:
      - POSTGRES_PASSWORD=${DB_PASSWORD}
      - POSTGRES_DB=appdb
```

```
# .env
DB_PASSWORD=supersecret
```

Challenge Code:

```
# docker-compose.yml
services:
  web:
    build: .
    ports:
      - "____:3000"
    environment:
      - DATABASE_URL=postgres://postgres:${____}@____:5432/appdb
    depends_on:
      - ____

  db:
    image: postgres:15-alpine
    environment:
      - POSTGRES_PASSWORD=${____}
      - POSTGRES_DB=appdb
```

```
# .env
DB_PASSWORD=____
```

03.0 Project Architecture with Compose

Learning Objectives:

- Describe the role of each file in a Dockerized project
- Explain how the `docker-compose.yml` file acts as the system's source of truth
- Understand how source code, Dockerfile, Compose file, and environment files relate to each other

Content Outline:

- The four key files in a Dockerized project: `Dockerfile`, `docker-compose.yml`, `.env`, `.dockerignore`
- How they relate: Dockerfile defines one service's image; Compose assembles the system; `.env` provides configuration; `.dockerignore` protects the build
- The `docker-compose.yml` as the entry point: a new developer can understand the entire system by reading it
- Named volumes: persisting database data across `docker compose down`
- Preview of Lesson 03.1: examining a complete project end to end

Transitions: This lesson frames the big picture. Lesson 03.1 walks through a complete realistic project and shows how all the pieces work together.

Key Principles:

- The `docker-compose.yml` is documentation as much as it is configuration
- Without named volumes, database data is lost every time containers are removed

Examples:

- Project directory layout:

```
myproject/
├── Dockerfile
├── docker-compose.yml
├── .env           ← gitignored
├── .dockerignore
└── src/
```

- Named volume: `volumes: db_data: /var/lib/postgresql/data` — data survives `docker compose down`

03.1 A Complete Dockerized Project

Learning Objectives:

- Read and interpret a real-world `docker-compose.yml` with multiple services
- Identify architectural decisions and explain the reasoning behind them

- Recognize when a project is correctly structured vs. when it has common setup problems

Content Outline:

- Walkthrough of a realistic project: web app + database + optional reverse proxy
- How services discover each other (default network, service name DNS)
- Named networks: when to define them explicitly and why
- Reading the compose file as an architecture diagram — what it reveals about the system
- Signs of a well-structured project vs. warning signs

Transitions: With the architecture understood, Section 04 assesses the full course.

Key Principles:

- The default Compose network is sufficient for most projects — named networks add value when you need network isolation between service groups
- A compose file that is easy to read is a system that is easy to operate

Examples:

- A three-service project: `web` (Node.js app), `db` (PostgreSQL), `proxy` (nginx) — each with a clear role, connected via the default network
 - Warning signs: services using `network_mode: host`, hardcoded passwords, no `.env` file, no `.dockerignore`
-

04.0 Putting It All Together

Learning Objectives:

- Consolidate understanding of Docker Compose across all course topics
- Connect concepts from each section into a coherent mental model
- Prepare for the scenario-based assessment

Content Outline:

- Review: the problem Compose solves and how it solves it
- Review: services, ports, networking, and environment variables
- Review: configuring a multi-service project and architectural best practices
- Common questions and clarifications before the assessment

Transitions: This lesson prepares students for the assessment. Lesson 04.1 tests understanding through scenario-based questions.

Key Principles:

- Compose is a tool for expressing system design as code — every key in the YAML has a reason
- The most common errors trace back to misunderstanding networking (`localhost` vs. service name) and missing configuration (env vars, `.dockerignore`)

Examples:

- The journey: from `docker run` chaos → a single `docker-compose.yml` that captures the whole system
-

04.1 Docker Compose Assessment 🧐

Learning Objectives:

- Demonstrate understanding of Docker Compose concepts, configuration, and architecture
- Apply knowledge to realistic scenarios a developer would encounter

Question Format: Scenario-based

Questions:

1. You have a `docker-compose.yml` with a `web` service and a `db` service. Your web app is trying to connect to the database using `localhost:5432` but keeps getting a connection refused error. What is the problem and what should the connection string be?
2. A teammate runs `docker compose up` and the web service starts before the database is ready, causing the app to crash. Which Compose key would you add to the `web` service definition to address this, and what does it actually guarantee?
3. You add a new `STRIPE_SECRET_KEY` to your `.env` file, but your running container doesn't see it. You didn't change `docker-compose.yml`. What do you need to do and why?
4. After running `docker compose down` and then `docker compose up`, all the data in your PostgreSQL database is gone. What Compose feature prevents this, and how do you configure it?
5. Your `docker compose up --build` is very slow. You suspect the build context is too large. What file do you check and what would you add to it?
6. You want to run your web app on port `8080` on your laptop but the app inside the container listens on port `3000`. How do you configure this in `docker-compose.yml`?
7. A new developer joins the team and needs to understand the entire system. Which single file should they read first and why?
8. You have two services that both try to map to host port `5000`. What error do you expect and how do you fix it?

Evaluation Criteria:

- Correct identification of the problem in each scenario
- Accurate use of Compose terminology (service name, port mapping, named volume, etc.)
- Practical fix proposed (not just identification of the issue)

Score Interpretation:

- 7-8 correct: Strong command of Docker Compose — ready to work with real multi-service projects
- 5-6 correct: Good foundation — review the sections covering the missed scenarios

- Below 5: Revisit Sections 01 and 02 before working with production Compose setups
-

05.0 What's Next

Content Outline:

- Course recap: what students accomplished
 - Understood why Docker Compose exists and what problem it solves
 - Learned the core concepts: services, ports, networking, environment variables
 - Wrote a complete `docker-compose.yml` for a real multi-service project
 - Understood how a well-structured Dockerized project is organized
- Connection to bigger picture: Compose is the foundation for local development environments that mirror production; the same patterns apply to Kubernetes and other orchestration tools
- Next steps:
 - Explore `docker compose watch` for hot-reloading during development
 - Learn about health checks and more sophisticated `depends_on` conditions
 - Investigate Docker volumes for sharing data between services
 - Look ahead to Skill 35: Debugging and optimizing the frontend
- Resources for further exploration:
 - Docker Compose official documentation
 - `docker compose --help` — explore commands you haven't used yet
 - Awesome Compose (GitHub) — real-world compose examples for popular stacks
- You've gone from running containers one at a time to orchestrating entire systems with a single file. That's a meaningful leap — systems thinking is one of the most valuable skills a developer can build.

Note: This is conclusion only — no theory, exercises, or assessment.
